

DRL 研究助理：用於自動化文獻回顧與實驗管理的 AI Harness 系統

課程：深度強化學習 — 作業四 日期：2026 年 5 月 格式：IEEE 論文格式

摘要

本報告介紹 **DRL 研究助理**，一個以 GPT-4o 作為核心控制器的 AI Harness 系統。系統透過 OpenAI function calling API 整合三個專用工具，自動化 DRL 研究中最耗時的三個瓶頸——文獻搜尋、實驗執行與結果分析。Agent 執行 ReAct (推理 + 行動) 迴圈，動態串接工具呼叫直到研究問題完整解決。本報告涵蓋系統架構、工具規格、workflow 設計與評量方法。

1. 問題定義與應用背景

DRL 研究者在探索新任務時，目前必須手動執行三個程序性步驟：(1) 瀏覽 ArXiv 搜尋相關文獻；(2) 配置 SB3 訓練腳本並等待實驗完成；(3) 彙整獎勵曲線、計算統計並得出結論。這些步驟不需要創意性科學判斷，非常適合由 **AI Harness** 自動化，讓研究者得以專注於更高層次的假設生成。

應用情境： 使用者輸入自然語言查詢 (如：「比較 PPO 和 DQN 在 CartPole 上訓練 30k 步」)，Agent 自主搜尋論文、執行實驗、分析結果，最後回傳包含論文引用、比較表與演算法建議的結構化 Markdown 報告。

2. AI 系統架構

系統採用三層架構：**LLM 控制器** → **工具層** → **記憶體層**。

LLM 控制器 (GPT-4o) 是系統的自主協調者，負責解析使用者意圖、規劃工具呼叫序列 (ReAct 推理)、評估工具輸出，並將所有中間結果綜合為最終回覆。system prompt 編碼領域知識 (DRL 演算法、Gymnasium 環境) 與工作流程慣例 (先搜尋 → 再實驗 → 最後分析)。

工具層 包含三個專用工具 (詳見第 3 章)，由 LLM 透過 OpenAI function calling API 動態呼叫。

記憶體層 分為兩層：

記憶體類型	實作方式	儲存內容
短期 (工作記憶)	LLM context window + <code>self.history</code> 清單	對話歷史、工具結果
長期 (持久記憶)	<code>agent/memory/experiment_db.json</code>	實驗結果、獎勵值、時間戳記

長期記憶獨立於 context window，確保跨 session 的實驗資料不會遺失，支援 `analyze_results(["all"])` 載入所有歷史紀錄。

3. 工具設計

工具一：`search_arxiv(query, max_results)`

屬性	說明
用途	從 ArXiv 擷取相關研究論文
API	ArXiv Atom Feed API (HTTP GET，無需驗證，使用 Python stdlib)
輸入	<code>query: str</code> (自由文字) ； <code>max_results: int</code> (預設 5)
輸出	<code>{papers: [{title, authors, abstract, url, published}], count}</code>
呼叫時機	工作流程第一步；每次呼叫暫停 3 秒以符合 rate limit

工具二：`run_rl_experiment(algorithm, environment, n_steps, hyperparams)`

屬性	說明
用途	訓練 SB3 Agent 並持久化結果
函式庫	Stable-Baselines3 + Gymnasium
輸入	<code>algorithm: "DQN" \ "PPO" \ "SAC"</code> ； <code>environment: str</code> ； <code>n_steps: int</code> ； <code>hyperparams: dict</code>
輸出	<code>{experiment_id (UUID), mean_reward, std_reward, status, timestamp}</code>
副作用	結果以 <code>experiment_id</code> 為鍵寫入 <code>experiment_db.json</code> ，供工具三後續讀取

呼叫範例：

```
{ "algorithm": "PPO", "environment": "CartPole-v1", "n_steps": 30000,
  "hyperparams": { "learning_rate": 3e-4, "clip_range": 0.2 } }
```

工具三：`analyze_results(experiment_ids, metrics)`

屬性	說明
用途	載入實驗結果、依表現排名並生成洞察
資料來源	<code>agent/memory/experiment_db.json</code>
輸入	<code>experiment_ids: list[str]</code> (指定 ID 或 <code>["all"]</code> 載入全部)

屬性	說明
輸出	{comparison_table (排序後), best_experiment, insights: list[str]}
呼叫時機	所有 run_rl_experiment 呼叫完成後

4. Agent Workflow 與 AI Orchestration

4.1 ReAct 多步驟流程

Agent 遵循 **ReAct** 迴圈：LLM 交替推理（文字）與行動（工具呼叫），持續到 `finish_reason = "stop"` 為止。以「比較 PPO 和 DQN，CartPole，30k 步」為例：

```

使用者查詢
↓
[LLM 推理] → 呼叫 search_arxiv("DQN PPO CartPole") → 取得 3 篇論文
↓
[LLM 推理] → 呼叫 run_rl_experiment(PPO, CartPole-v1, 30000) → mean_reward: 500.0
↓
[LLM 推理] → 呼叫 run_rl_experiment(DQN, CartPole-v1, 30000) → mean_reward: 9.7
↓
[LLM 推理] → 呼叫 analyze_results(["9253aa26", "be006dbf"]) → 比較表 + 洞察
↓
[LLM 綜合] → 回傳 Markdown 報告 (引用 + 比較表 + 建議)

```

4.2 Orchestration 機制

OpenAI function calling API 是 orchestration 的核心骨幹。每個工具以 JSON schema 定義，LLM 透過回傳 `finish_reason = "tool_calls"` 選擇工具，Python harness 分派執行後以 `role: "tool"` + `tool_call_id` 回傳結果，並重送完整歷史給 LLM 繼續推理。

LLM 自主進行三類決策：**工具選擇**（文獻不足→search；實驗未執行→run；需比較→analyze）、**參數推斷**（從使用者意圖推斷 n_steps、environment）、**終止判斷**（資訊充足時停止呼叫工具）。

Orchestration 屬性	機制
工具排序	system prompt 規定「先搜尋 → 再實驗 → 最後分析」
錯誤回復	工具回傳 <code>status: "failed"</code> 時，LLM 診斷並重試或降級
多輪記憶	<code>self.history</code> 累積全部輪次，LLM 可跨輪參照 experiment ID
跨 session	<code>experiment_db.json</code> 持久保存，重啟後仍可載入歷史

5. 評量方法

本系統非純 RL 任務，需多維度評量。

維度	指標	量測方式	目標值
功能正確性	工具呼叫成功率	回傳 <code>status: "completed"</code> 比例	≥ 95%
	實驗可重現性	3 次相同實驗的標準差	≤ 平均值 10%
	論文擷取相關性	人工評分 (1–5)	≥ 4.0
Orchestration 品質	每次查詢工具呼叫次數	統計 20 個基準查詢	3–5 次
	計畫正確性	是否先搜尋再實驗	≥ 90%
	冗餘工具呼叫	無新資訊的重複呼叫	≤ 1 次
輸出品質	洞察準確性	對照實驗排名真實結果	≥ 90% 吻合
	引用準確性	核對 ArXiv 真實資料	≥ 95% 正確
系統效能	端對端延遲 (不含訓練)	搜尋 + 分析查詢	≤ 10 秒
	API 成本	GPT-4o 計費代幣	≤ \$0.05/session

實驗驗證結果 (Live Demo)：以查詢「比較 PPO 和 DQN · CartPole · 30k 步」進行端對端測試。PPO 達到完美獎勵 **500.0 ± 0.0** (30k 步收斂) ；DQN 僅 **9.7 ± 1.55** (步數不足，Q 值不穩定)。工具呼叫共 3 次 (無冗餘)，計畫正確性與洞察準確性皆達標。ArXiv rate limit 觸發時，GPT-4o 優雅降級為引用內部知識，展示系統韌性。

參考文獻

1. Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*.
2. Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
3. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518.
4. Raffin, A., et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*.
5. OpenAI (2025). Function calling. *OpenAI Platform Documentation*.